Εργασία Εργαστηρίου ΛΣ (2025-2026)

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: ΠΑΣΧΑΛΗΣ ΚΟΥΚΟΣ

ΑΜ : ΙΤ2024041

Περιγραφή τρόπου με τον οποίο θα εργαστώ:

Θα εργαστώ σε WSL (Windows Subsystem for Linux) , διότι το VM (Virtual Box έχω προσωπικά) κολλάει αρκετά. Το WSL είναι μια δυνατότητα των Windows που επιτρέπει στους χρήστες να τρέχουν ένα πλήρες περιβάλλον Linux απευθείας μέσα στα Windows, χωρίς να χρειάζεται να εγκαταστήσουν ξεχωριστό λειτουργικό σύστημα ή να χρησιμοποιήσουν εικονική μηχανή , η οποία μπορεί να κολλάει, όπως στη περίπτωση μου. Θα εργαστώ μέσα στο directory 'OS' , το οποίο δημιούργησα αποκλειστικά για την εργασία μας. Τα προγράμματα C θα τα γράψω στο IDE Clion (JETBRAINS). Ακολουθεί η υλοποίηση χωρισμένη σε μέρη και με αρκετά screenshots αλλά και επεξηγήσεις , για να γίνει κατανοητός ο τρόπος σκέψης-εκτέλεσης που έχω.

ΜΕΡΟΣ Α – Προετοιμασία περιβάλλοντος & βασικές εντολές

1. Απάντηση ερωτήματος :

```
pashalis@pashalis:~/OS$ mkdir -p monitor/raw monitor/processed monitor/reports
pashalis@pashalis:~/OS$ ls
monitor
pashalis@pashalis:~/OS$ cd monitor
pashalis@pashalis:~/OS/monitor$ ls
processed  raw  reports
pashalis@pashalis:~/OS/monitor$
```

Με την εντολή mkdir δημιουργώ καταλόγους και με το flag -p φτιάχνω ενδιάμεσους φακέλους αν δεν υπάρχουν με 1 εντολή. Με την ls βλέπω το περιεχόμενο του τρέχοντος μονοπατιού στο οποίο βρίσκομαι (στη περίπτωση μας του τρέχοντος καταλόγου) ενώ με την cd αλλάζω απόλυτο ή σχετικό μονοπάτι (στη περίπτωσή μας εδώ σχετικό). Θα χρησιμοποιώ συχνά την ls , έτσι ώστε να επιβεβαιώνω αυτά που κάνω.

2. Απάντηση ερωτήματος :

```
pashalis@pashalis:~/OS$ cd monitor/raw
pashalis@pashalis:~/OS/monitor/raw$ touch system.log network.log security.log
pashalis@pashalis:~/OS/monitor/raw$ ls
network.log  security.log  system.log
pashalis@pashalis:~/OS/monitor/raw$
```

Με την εντολή touch δημιουργώ 3 αρχεία σύμφωνα με το ζητούμενο της άσκησης.

3. Απάντηση ερωτήματος :

```
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - System started" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "Backup FAILED" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "CRITICAL: High temperature" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - Update completed" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "IP address: 192.168.1.1" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "ERROR: Disk full" >> system.log
pashalis@pashalis:~/OS/monitor/raw$ echo "ERROR: Memory leak" >> system.log
```

Ξεκινάμε από το αρχείο system.log και βάζουμε σχετικά μηνύματα με το ΛΣ και γενικά το σύστημά μας χρησιμοποιώντας την εντολή echo “message” >> system.log

```
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - Network active" >> network.log
pashalis@pashalis:~/OS/monitor/raw$ echo "ERROR: Connection lost" >> network.log
pashalis@pashalis:~/OS/monitor/raw$ echo "Ping to 10.100.0.10 OK" >> network.log
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - Router rebooted" >> network.log
pashalis@pashalis:~/OS/monitor/raw$ echo "DHCP FAILED" >> network.log
pashalis@pashalis:~/OS/monitor/raw$ echo "CRITICAL: VPN disconnected" >> network.log
```

Με ίδια λογική συνεχίζω με μηνύματα σχετικά με το διαδίκτυο μέσα στο αρχείο network.log

```
pashalis@pashalis:~/OS/monitor/raw$ echo "ERROR: Login failed" >> security.log
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - Security scan for viruses started" >> security.log
pashalis@pashalis:~/OS/monitor/raw$ echo "Blocked IP: 172.16.1.1" >> security.log
pashalis@pashalis:~/OS/monitor/raw$ echo "Password change FAILED" >> security.log
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - Scan completed" >> security.log
pashalis@pashalis:~/OS/monitor/raw$ echo "2025-12-23 - No viruses detected" >> security.log
```

Με ίδια λογική για το security.log με μηνύματα σχετικά με την ασφάλεια του υπολογιστή

4. Απάντηση ερωτήματος :

```
pashalis@pashalis:~/OS/monitor/raw$ ls -al
total 20
drwxr-xr-x 2 pashalis pashalis 4096 Dec 23 14:32 .
drwxr-xr-x 5 pashalis pashalis 4096 Dec 23 14:26 ..
-rw-r--r-- 1 pashalis pashalis  142 Dec 23 15:08 network.log
-rw-r--r-- 1 pashalis pashalis  174 Dec 23 15:14 security.log
-rw-r--r-- 1 pashalis pashalis  159 Dec 23 14:57 system.log
```

Εμφάνιση αρχείων αναλυτικά με την ls -al

```
pashalis@pashalis:~/OS/monitor/raw$ wc *.log
  6  20 142 network.log
  6  25 174 security.log
  7  22 159 system.log
 19  67 475 total
```

Συνολικά στα 3 αρχεία έχουμε 19 γραμμές (6+6+7)=19 γραμμές. Η εντολή ‘wc *.log’ μας δείχνει γραμμές, λέξεις και τέλος χαρακτήρες κάθε αρχείου που έχει κατάληξη .log (στη περίπτωσή μας των τριών αρχείων μας). Για να δούμε μόνο γραμμές μπορούμε να γράψουμε ‘wc -l *.log’.

ΜΕΡΟΣ Β – Φιλτράρισμα logs με Regular Expressions

1. Απάντηση ερωτήματος:
   ->Γραμμές που ξεκινούν με ημερομηνία (pattern: YYYY-MM-DD):

```
pashalis@pashalis:~/OS/monitor/raw$ grep -E '^[[:digit:]]{4}-[[:digit:]]{2}-[[:digit:]]{2}' *.log
network.log:2025-12-23 - Network active
network.log:2025-12-23 - Router rebooted
security.log:2025-12-23 - Security scan for viruses started
security.log:2025-12-23 - Scan completed
security.log:2025-12-23 - No viruses detected
system.log:2025-12-23 - System started
system.log:2025-12-23 - Update completed
pashalis@pashalis:~/OS/monitor/raw$
```

Χρησιμοποιώ grep -E: Η παράμετρος-E σημαίνει Extended Regular Expressions (ERE) για όλα τα αρχεία που τελειώνουν σε .log. Το ^ σημαίνει ότι 'ξεκινάει με..' 4 ψηφία-2 ψηφία-2 ψηφία.

->Γραμμές που περιέχουν είτε τις λέξεις ERROR, FAILED, CRITICAL:

```
pashalis@pashalis:~/OS/monitor/raw$ grep -E 'ERROR|FAILED|CRITICAL' *.log
network.log:ERROR: Connection lost
network.log:DHCP FAILED
network.log:CRITICAL: VPN disconnected
security.log:ERROR: Login failed
security.log:Password change FAILED
system.log:Backup FAILED
system.log:CRITICAL: High temperature
system.log:ERROR: Disk full
system.log:ERROR: Memory leak
pashalis@pashalis:~/OS/monitor/raw$
```

Το '|' σημαίνει 'ή'. Περιέχει είτε την λέξη ERROR ή CRITICAL ή FAILED για όλα τα αρχεία .log

->Γραμμές που περιέχουν IPv4 διεύθυνση:

```
pashalis@pashalis:~/OS/monitor/raw$ grep -E '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' *.log
network.log:Ping to 10.100.0.10 OK
security.log:Blocked IP: 172.16.1.1
system.log:IP address: 192.168.1.1
pashalis@pashalis:~/OS/monitor/raw$
```

Θεωρούμε ότι υπάρχουν μόνο σωστές IP addresses. Σε αυτή τη περίπτωση λέμε 'από το 0 έως το 9 να εμφανιστεί από 1 έως 3 φορές. από το 0 έως το 9 να εμφανιστεί από 1 έως 3 φορές. από το 0 έως το 9 να εμφανιστεί από 1 έως 3 φορές. από το 0 έως το 9 να εμφανιστεί από 1 έως 3 φορές'. Αυτό μας εμφανίζει τις IP χωρίς έλεγχο εγκυρότητας. Θεωρούμε ότι οι τιμές του συστήματος είναι σωστές και δε χρειάζεται έλεγχος.

2. Απάντηση ερωτήματος:

```
pashalis@pashalis:~/OS/monitor/raw$ grep -E '^[[:digit:]]{4}-[[:digit:]]{2}-[[:digit:]]{2}' *.log > ../processed/alerts.raw
pashalis@pashalis:~/OS/monitor/raw$ grep -E 'ERROR|FAILED|CRITICAL' *.log >> ../processed/alerts.raw
pashalis@pashalis:~/OS/monitor/raw$ grep -E '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' *.log >> ../processed/alerts.raw
pashalis@pashalis:~/OS/monitor/raw$ cat ../processed/alerts.raw
network.log:2025-12-23 - Network active
network.log:2025-12-23 - Router rebooted
security.log:2025-12-23 - Security scan for viruses started
security.log:2025-12-23 - Scan completed
security.log:2025-12-23 - No viruses detected
system.log:2025-12-23 - System started
system.log:2025-12-23 - Update completed
network.log:ERROR: Connection lost
network.log:DHCP FAILED
network.log:CRITICAL: VPN disconnected
security.log:ERROR: Login failed
security.log:Password change FAILED
system.log:Backup FAILED
system.log:CRITICAL: High temperature
system.log:ERROR: Disk full
system.log:ERROR: Memory leak
network.log:Ping to 10.100.0.10 OK
security.log:Blocked IP: 172.16.1.1
system.log:IP address: 192.168.1.1
pashalis@pashalis:~/OS/monitor/raw$
```

Αποθήκευση αποτελεσμάτων στο αρχείο alerts.raw . Χρησιμοποιώ αρχικά '>' το οποίο λέει ουσιαστικά 'πάρε αυτό το αποτέλεσμα κι βάλτο στο αρχείο alerts.raw'. Στη συνέχεια χρησιμοποιώ '>>' ώστε να κάνει append κι να κρατήσει τις προηγούμενες τιμές του αρχείου και να μην τις σβήσει. Δε θέλουμε Override. Πρέπει να αποθηκεύσω όλα τα αποτελέσματα. Με την 'cat' εμφανίζω το περιεχόμενο του αρχείου μας. Κάνω ../ ώστε να βγει από τον κατάλογο raw και να βρει τον κατάλογο processed.

3. Απάντηση ερωτήματος:

```
pashalis@pashalis:~/OS/monitor/processed$ sort alerts.raw | uniq > alerts.sorted
pashalis@pashalis:~/OS/monitor/processed$ ls
alerts.raw  alerts.sorted
```

Αλλάζω κατάλογο και μπαίνουμε στον processed. Με την 'sort' κάνω ταξινόμηση τα περιεχόμενα του αρχείου κι όχι το ίδιο το αρχείο. Στη συνέχεια η 'uniq' παίρνει τα ταξινομημένα στοιχεία μέσω pipe '|',δηλαδή λαμβάνει το αποτέλεσμα της sort, κάνει αφαίρεση διπλότυπων γραμμών και τέλος, αποθηκεύεται το συνολικό αποτέλεσμα στο αρχείο alerts.sorted (Στο δικό μας αρχείο δεν έβαλα στο Μέρος Α διπλότυπα. Επομένως δε θα υπάρχουν αφαιρέσεις στη περίπτωσή μας)

ΜΕΡΟΣ Γ – Συνδυασμός Pipes & Redirection για δημιουργία αναφοράς

1-2. Απάντηση ερωτημάτων με 1 γραμμή-εντολή :

```
pashalis@pashalis:~/OS/monitor$ cat processed/alerts.sorted | awk '
> BEGIN {total_alerts = 0; errors = 0; local_network_events = 0 }
> {
> if (/ERROR/) errors++
> if (/192\.168\./) local_network_events++
>  total_alerts++
> }
>  END { print "Total Alerts: " total_alerts ", Errors: " errors ", Local Network Events: " local_network_events }' > reports/daily_summary.txt
pashalis@pashalis:~/OS/monitor$
```

Για να απαντήσω σε αυτή την ερώτηση βρήκα επιπλέον πληροφορίες για την awk (για το if της) πέρα από το εργαστήριο στον σύνδεσμο :

https://www.geeksforgeeks.org/linux-unix/awk-command-unixlinux-examples/

Επεξήγηση : Με την εντολή 'cat processed/alerts.sorted' ανοίγω το περιεχόμενο το alerts.sorted κι στη συνέχεια το διοχετεύω μέσω pipe '|' στην εντολή 'awk'. Η εντολή αυτή είναι μια γλώσσα προγραμματισμού σχεδιασμένη να εντοπίζει συγκεκριμένα πρότυπα και να εφαρμόζει σε αυτά κάποιες ενέργειες. Με αυτή τη λογική λοιπόν αρχικοποιώ τις τρείς μεταβλητές – μετρητές μου στο 0 στο BEGIN. Στη 'main' {} αρχικά ελέγχω αν στη τρέχουσα γραμμή που ψάχνει περιέχεται η λέξη ERROR κι αν ναι, αυξάνω τον αντίστοιχο μετρητή κατά 1. Το ίδιο για την διεύθυνση 192.198.* και τέλος, θεωρώ ότι το total alerts αποτελεί κάθε γραμμή στο αρχείο alerts.sorted.Ότι όλα δηλαδή είναι total alerts. (ΔΙΕΥΚΡΊΝΙΣΗ) Επειδή με μπέρδεψε λίγο το κομμάτι αυτό , αν total alerts θεωρούνται οι γραμμές μόνο με FAILED,ERROR,CRITICAL τότε έπρεπε να γράψω if (/ERROR|FAILED|CRITICAL/) total_alerts++ όπου '|' σημαίνει 'ή' . Στο END κάνω την εκτύπωση που ζητείται κι με τη βοήθεια του redirection αποθηκεύω το αποτέλεσμα στο αρχείο daily_summary.txt στον κατάλογο reports. Περιεχόμενο daily_summary.txt :

```
pashalis@pashalis:~$ cat OS/monitor/reports/daily_summary.txt
Total Alerts: 19, Errors: 4, Local Network Events: 1
```

ΜΕΡΟΣ Δ – Διαχείριση Διεργασιών

1. Εκτελέστε μια εντολή που "τρέχει στο παρασκήνιο (background)" και γράφει περιοδικά timestamps σε ένα αρχείο raw/timestamps.log**.** Απάντηση :
   Πηγή πληροφοριών για το while loop bash που χρησιμοποιήθηκε:
   https://www.cyberciti.biz/faq/bash-while-loop/

```
pashalis@pashalis:~/OS/monitor$ while true; do date "+%Y-%m-%d %H:%M:%S" >> raw/timestamps.log; sleep 5; done &
[1] 465
pashalis@pashalis:~/OS/monitor$
```

   Ουσιαστικά χρησιμοποιώ ένα ατέρμων loop που γράφει κάθε 5 δευτερόλεπτα timestamps στο αρχείο timestamps.log μέσα στον κατάλογο raw.Στέλνω τη διεργασία αυτή να εκτελείται στο background με το σύμβολο &. Κάνω append με redirection >> για να συγκρατώ κάθεtimestamp . Μας δείχνει ήδη το PID του πατέρα που είναι ίσο με 465.

2.

```
pashalis@pashalis:~/OS/monitor$ ps -a
    PID TTY          TIME CMD
    431 pts/1    00:00:00 bash
    465 pts/0    00:00:00 bash
    486 pts/0    00:00:00 sleep
    487 pts/0    00:00:00 ps
pashalis@pashalis:~/OS/monitor$ ps -a | grep 'bash'
    431 pts/1    00:00:00 bash
    465 pts/0    00:00:00 bash
pashalis@pashalis:~/OS/monitor$
```

Βλέπουμε όλες τις διεργασίες με το ps -a. Παρατηρώ αυτό που περιμέναμε. Η διεργασία μας με PID 465 έχει πάρει όνομα bash και την βρίσκω με ps -a -> αποτέλεσμα μέσω pipe '|' στο grep. Από εδώ βγάζουμε και χρήσιμα συμπεράσματα για την ανασκόπηση στο τέλος

3.

```
pashalis@pashalis:~/OS/monitor$ renice 5 465
465 (process ID) old priority 0, new priority 5
pashalis@pashalis:~/OS/monitor$
```

Renice -n 5 <PID> ή απλά renice 5 <PID> αλλάζω την προτεραιότητα από 0 σε 5. Βάζω δηλαδή χαμηλότερη προτεραιότητα.

4.

```
pashalis@pashalis:~/OS/monitor$ kill -TERM 465
pashalis@pashalis:~/OS/monitor$ ps -a
    PID TTY          TIME CMD
    431 pts/1    00:00:00 bash
    534 pts/0    00:00:00 ps
[1]+  Terminated              while true; do
    date "+%Y-%m-%d %H:%M:%S" >> raw/timestamps.log; sleep 5;
done
pashalis@pashalis:~/OS/monitor$
```

Επιτυχής ασφαλής τερματισμός με σήμα SIGTERM , χωρίς να χρειαστεί το SIGKILL.Παρατηρούμε ότι η διεργασία μας πράγματι τερματίστηκε (terminated).

5. Βλέπουμε ότι η διεργασία μας πήρε PID = 465 κι εμφανίζεται ως bash στο σύστημά μας. Αυτός είναι ο πατέρας πολλών θυγατρικών διεργασιών 'sleep' , οι οποίες ζουν για 5 δευτερόλεπτα, πεθαίνουν κι στη συνέχεια παίρνουν διαφορετικό PID αυξανόμενο , διότι ξαναγεννιούνται από τον πατέρα που ουσιαστικά : γράφει(δημιουργία sleep), κοιμάται για 5 δευτερόλεπτα, ξαναγράφει (δημιουργία ξανά sleep(με διαφορετικό PID)), κοιμάται για 5 δευτερόλεπτα .

bash (PID 465 πατέρας, σταθερό)

⬇ fork()

sleep (PID π.χ. 486, ζει 5 δευτερόλεπτα) -> πεθαίνει

⬇ wait()

bash (PID 465 πατέρας δημιουργεί ξανά)

⬇ (ξανά δημιουργεί)

sleep (PID πχ 486+x, ζει 5 δευτερόλεπτα) -> πεθαίνει

...κλπ... μέχρι να κάνω termination

ΜΕΡΟΣ Ε – Υλοποίηση προγράμματος C για ανάλυση logs

Πρόγραμμα :

```
// 2024041 Πασχάλης Κούκος
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#define BUFFER_SIZE 1024
void error(const char *msg);
```

```c
void print_message(int lines,int digits, int errors);
void line_change(char *buffer,int i,int flag_digits, int *lines, int
*digits, int *errors);

int main(int argc, char *argv[]) { // argv[0] είναι το file name.c κι
argv[1] πρέπει να είναι το αρχείο .log ( άρα σύνολο 2 )
    //πρέπει argc == 2
    if (argc != 2) {fprintf(stderr,"File not provided\n");exit(1);} //
αν δε δώσει αρχείο -> μήνυμα σφάλματος κι έξοδος
    int fd = open(argv[1], O_RDONLY); //το ανοίγω μόνο για ανάγνωση
    if (fd==-1) {error("open"); exit(1);} //σφάλμα ανοίγματος

    char c;
    ssize_t b = read(fd, &c, 1); //διάβασμα ενός χαρακτήρα (αν υπάρχει)
για έλεγχο

    if (b == 0) {
        // το αρχείο είναι κενό - δε διαβάστηκε χαρακτήρας
        puts("Empty file");
        close(fd); //κλείσιμο για ασφάλεια
        exit(2); //κενό αρχείο
    }
    if (b == -1) {
        // σφάλμα ανάγνωσης
        error("read");
        close(fd);
        exit(1);
    }
    lseek(fd, 0, SEEK_SET); // επιστροφή στην αρχή του αρχείου

    int lines = 0;int line_errors = 0;int line_with_digits = 0;
    char buffer[BUFFER_SIZE];int flag_digits = 0;int i =0;
    while (b > 0) { //οσο έχει μέσα bytes να διαβάσει
        if (isdigit(c)) {
            flag_digits=1;
        }

        if (i<BUFFER_SIZE-1 && c!='\n')buffer[i++]=c; // αν δεν είναι
αλλαγή γραμμής και δεν ξεπερνά το μαξ μέγεθος προσθέτω
        //ώστε να ολοκληρώσει μια γραμμή
        if (c == '\n') { //αλλαγή γραμμής

line_change(buffer,i,flag_digits,&lines,&line_with_digits,&line_errors);
            //επαναφορά για επόμενη επανάληψη
            flag_digits = 0;
            i=0;
        }
        b = read(fd, &c, 1); //διάβασμα επόμενου χαρακτήρα
    }
    if (i>0) { // για να μη μου ξεφύγει καμία γραμμή - έχει χαρακτήρες
στο buffer
        // μπορεί μια γραμμή να μη τελειώνει σε \n αλλά πχ με EOF
        // αν υπάρχουν χαρακτήρες στο buffer τους επεξεργαζόμαστε ως
γραμμή

line_change(buffer,i,flag_digits,&lines,&line_with_digits,&line_errors);
    }
    print_message(lines,line_with_digits,line_errors);
    close(fd);
    return 0; // επιτυχία
}
```

```c
void error(const char *msg) { //διαχείριση με p error
    perror(msg);
}

void print_message(const int lines,const int digits,const int errors) {
    // εμφάνιση μηνυμάτων στο τέλος
    printf("Total lines: %d\nTotal lines with word 'ERROR': %d\nTotal
lines with digits: %d\n",lines, errors, digits);
}

void line_change(char *buffer,int i,int flag_digits, int *lines, int
*digits, int *errors) { // συνάρτηση για την αλλαγή γραμμής
    buffer[i]='\0'; // Τέλος, γραμμής - string
    (*lines)++; //αυξάνω γραμμή
    if (flag_digits) {
        (*digits)++; //περιέχει ψηφίο
    }
    if (strstr(buffer, "ERROR")!=NULL) {
        (*errors)++; // το 'ERROR' είναι substring του ολόκληρου string-
γραμμής
    }
}
```

Φρόντισα να προσθέσω πολλά σχόλια, έτσι ώστε να είναι πλήρως κατανοητός ο τρόπος επίλυσης που σκέφτηκα. Ακολουθεί μια σύντομη περιγραφή του κώδικα:

Το πρόγραμμα διαβάζει ένα αρχείο καταγραφής από το cmd (`.log`) γραμμικά, χρησιμοποιώντας buffer για να αποθηκεύει κάθε γραμμή κι έπειτα να κάνει τις απαραίτητες συγκρίσεις. Κρίσιμα σημεία: 1) Έλεγχος ύπαρξης και περιεχομένου αρχείου πριν την επεξεργασία του κι διαχείριση με perror-errno, 2) Αυτόματη αύξηση του counter γραμμών με τη συνάρτηση `line_change()`, η οποία αυξάνει τις γραμμές , αν έχει πάρει τιμή 1 (true) η flag_digits (που χρησιμοποιεί την isdigit()) τότε αυξάνω μετρητή γραμμών με αριθμούς και τέλος ανιχνεύει την υποσυμβολοσειρά "ERROR" μέσα στον buffer μας(χρησιμοποιώντας την strstr(), η οποία εντοπίζει αν ένα substring – στη περίπτωσή μας το 'ERROR' – περιέχεται μέσα σε ένα γενικό string -στη περίπτωσή μας μια ολόκληρη γραμμή-), 3) Διαχείριση γραμμών που τερματίζουν είτε με `\n` είτε με EOF είτε με οτιδήποτε άλλο. 4) Εκτύπωση μηνύματος με το αποτέλεσμα. Παράδειγμα εκτέλεσης μαζί με επιβεβαίωση με cat:

```
pashalis@pashalis:~/OS/monitor$ gcc -o  analyze_log analyze.c
pashalis@pashalis:~/OS/monitor$ ./analyze_log raw/network.log
Total lines: 6
Total lines with word 'ERROR': 1
Total lines with digits: 3
pashalis@pashalis:~/OS/monitor$ cat raw/network.log
2025-12-23 - Network active
ERROR: Connection lost
Ping to 10.100.0.10 OK
2025-12-23 - Router rebooted
DHCP FAILED
CRITICAL: VPN disconnected
pashalis@pashalis:~/OS/monitor$
```

ΜΕΡΟΣ ΣΤ – Shell Script που αυτοματοποιεί όλο το σύστημα

Απάντηση .sh :

```
#!/bin/bash

DIR="$1" #δέχεται ως όρισμα έναν φάκελο logs (π.χ. raw/) (1)

if [ $# -ne 1 ] # αν το πλήθος των ορισμάτων != 1
then
    echo "Illegal number of arguments.Required: 1 , Given: $#"
    exit 1
fi # ελέγχει ότι έχει δοθεί 1 μόνο όρισμα , όπως πρέπει (2)

if [ ! -d "$DIR" ]
then
    echo "Directory '$1' does not exist"
    exit 1
fi #ελέγχει ότι ο κατάλογος που δόθηκε υπάρχει (3)

#εμφανίζει το όνομα του script και τον συνολικό αριθμό ορισμάτων (4)
echo "name of script is: '$0'"
echo "number of arguments provided are: $#"

# παρακάτω ακολουθεί υλοποίηση του (5) κι του (6)
# καλεί το πρόγραμμα analyze_log για κάθε αρχείο στο directory
# συγκεντρώνει όλα τα αποτελέσματα σε ένα νέο αρχείο:
reports/full_report.txt

FILE_TO_SAVE="reports/full_report.txt" # αρχείο που θα αποθηκεύσω το
αποτέλεσμα
touch "$FILE_TO_SAVE" #δημιουργία αρχείου

echo -n "Do you want to start the process?"
read answer

while [ "$answer" = "y" ]
do
    echo "" > "$FILE_TO_SAVE"   # Δημιουργία / καθαρισμός report απο
προηγούμενα πριν την επεξεργασία
    for file in "$DIR"/*.log # για κάθε αρχείο .log στον φάκελο που μας
έδωσε
    do

            # Categorization με case
            case "$file" in
                *system*)
                        category="SYSTEM"
                        ;;
                *network*)
                        category="NETWORK"
```

```
                        ;;
                *security*)
                        category="SECURITY"
                        ;;
                *)
                    category="$file" #ολόκληρο το όνομα πχ
raw/one_report.log
                        ;;
            esac

            # Κλήση analyze_log και ασφαλής ανάγνωση/διάσπαση γραμμών με
IFS(5)
            ./analyze_log "$file" | while IFS= read -r string_line
            do
                echo "[$category] > $string_line" >> "$FILE_TO_SAVE" # κάνω
append την γραμμή στο reports/full_report.txt (6)
            done
    done
    echo "Report completed successfully at: $FILE_TO_SAVE" #επιτυχία
    echo -n "Do you want to restart the process?"
    read answer
done
```

Επεξήγηση σκέψης κι εκτέλεσης :

Αρχικά δέχομαι το πρώτο όρισμα που μου δίνει ο χρήστης κι το αποθηκεύω στην μεταβλητή DIR με “” ώστε να πάρει όλο το όνομα. Π.χ. αν το όνομα του directory είναι ‘my files’ κι δεν χρησιμοποιήσω τα ‘αυτάκια’ , τότε θα πάρει μόνο το ‘my’ κι θα προκύψει σφάλμα. Επομένως χρησιμοποιώ παντού για ασφάλεια τα “”. Στη συνέχεια ελέγχω αν το πλήθος των ορισμάτων not equal με το 1 κι αν συμβεί αυτό, τερματίζω το script με αντίστοιχο μήνυμα. Έπειτα ελέγχω αν το όρισμα που μας έδωσε ο χρήστης αντιστοιχεί σε φάκελο με το –(d)irectory και αν όχι,τερματίζω με κατάλληλο μήνυμα ότι δεν υπάρχει ο φάκελος. Εμφανίζω πλήθος args καθώς και το όνομα του αρχείου μας ολόκληρο. Μετά από την αρχική διαδικασία ακολουθεί η ανάλυση. Δημιουργώ αρχείο με touch reports/full_report.txt ώστε να αποθηκεύσω το αποτέλεσμα της ανάλυσης, αν και η δημιουργία γίνεται κι αργότερα με το echo αυτόματα,αλλά εγω προσθέτω το touch (αν και ίσως αχρείαστο) για να είναι πιο ξεκάθαρο. Διαβάζω από τον χρήστη απάντηση κι αν θέλει να πατήσει “y” ,τότε ξεκινάμε. Μπαίνουμε στο while μέχρις ότου ο χρήστης να πατήσει οτιδήποτε άλλο εκτός από “y” (Θα ρωτάω επαναληπτικά τον χρήστη μόλις τελειώνει κάθε ανάλυση αν επιθυμεί επανάληψή της).Πριν το for καθαρίζω το περιεχόμενο του αρχείου, διότι τώρα θα το ενημερώσουμε με καινούργια δεδομένα. Για κάθε αρχείο μέσα στο φάκελό μας που τελειώνει σε .log βρες με case την κατηγορία του κι αν δεν είναι μια από τις 3 γνωστές (SYSTEM/NETWORK/SECURITY) τότε βάλε το όνομα του τρέχοντος αρχείου ως κατηγορία (έξτρα πληροφορίες για το case βρήκα στο https://linuxize.com/post/bash-case-statement/). Τέλος, καλώ το πρόγραμμά μας το εκτελέσιμο αρχείο του analyze_log με το όνομα του τρέχοντος αρχείου κι το αποτέλεσμα το περνάω με pipe ‘|’ στο IFS(έξτρα πληροφορίες διάβασα στο https://www.tutorialspoint.com/the-meaning-of-ifs-in-bash-scripting-on-linux) , το οποίο διασφαλίζει μαζί με το read -r ότι θα έχω σωστή διάσπαση γραμμών->διαβάζω ολόκληρη τη γραμμή όπως ακριβώς είναι(το -r δεν αγνοεί τα backslashes, διαβάζει την γραμμή ακριβώς

όπως είναι) κι από εκεί το κάνω append (>>) με redirection στο reports/full_report.txt. Ρωτάω αν θέλει να το επαναλάβει κι αν όχι, τερματίζει η διαδικασία. Παράδειγμα εκτέλεσης στο terminal :

```
pashalis@pashalis:~/OS/monitor$ ./run_monitor.sh raw
name of script is: './run_monitor.sh'
number of arguments provided are: 1
Do you want to start the process?y
Report completed successfully at: reports/full_report.txt
Do you want to restart the process?y
Report completed successfully at: reports/full_report.txt
Do you want to restart the process?n
pashalis@pashalis:~/OS/monitor$
```

Διαχείριση εισόδου με σφάλμα :

```
pashalis@pashalis:~/OS/monitor$ ./run_monitor.sh nodir
Directory 'nodir' does not exist
pashalis@pashalis:~/OS/monitor$ ./run_monitor.sh nodir other
Illegal number of arguments.Required: 1 , Given: 2
pashalis@pashalis:~/OS/monitor$
```

ΜΕΡΟΣ Ζ – Threads για παράλληλη ανάλυση logs

```c
// 2024041 Πασχάλης Κούκος
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <pthread.h>
#include <ctype.h>
#define FILESIZE 256
#define BUFFER_SIZE 1024

typedef struct {
    char filename[FILESIZE];    // ποιο αρχείο θα αναλύσει
    int lines;              // πόσες γραμμές
    int errors;             // πόσα errors
    int digits;             //πόσες γραμμές περιέχουν digit
} file_thread;

void line_change(char *buffer,int i,int flag_digits, int *lines, int
*digits, int *errors);
void error(const char *msg);
void* analyze(void* arg);
void print_each_log_stats(file_thread *file,int *lines,int *errors,int
*digits);

int main(int argc, char *argv[]) {
    if (argc < 2) {fprintf(stderr,"File not provided\n");exit(1);}
//σφάλμα, κανένα όρισμα
    file_thread file_threads[argc-1]; //πίνακας με τα αρχεία μεγέθους
ίσου με τα ορίσματα που μας έδωσε, δηλαδή τα αρχεία
    pthread_t thread[argc-1]; // πίνακας με τα threads μας

    for (int i=0; i<argc-1; i++) {
```

```
        strcpy(file_threads[i].filename,argv[i+1]); // βάζω το όνομα του
αρχείου
        file_threads[i].digits=0; file_threads[i].errors=0;
file_threads[i].lines=0; //αρχικοποίηση σε 0
        if (pthread_create(&thread[i], NULL, analyze, &file_threads[i])
!= 0) { //δημιουργία thread για κάθε αρχείο
            //έλεγχος
            error("pthread_create");
            exit(1);
        }
    }

    for (int i = 0; i < argc - 1; i++) {
        if (pthread_join(thread[i], NULL) != 0) { //για κάθε αρχείο
κάνει join, ολοκληρώνεται η δουλειά και συνεχίζει με το επόμενο
            //Έλεγχος
            error("pthread_join");
            exit(1);
        }
    }

    int total_lines = 0, total_errors = 0, total_digits = 0; //μετρητές
για σύνολο
    for (int i = 0; i < argc - 1; i++) {

print_each_log_stats(&file_threads[i],&total_lines,&total_errors,&total_
digits); //εκτυπώνει stats για 1 αρχείο
        //και προσθέτει για το σύνολο
    }
    //εκτύπωση συνολικών στατιστικών
    printf("    TOTAL LINES: %d\n   TOTAL ERRORS: %d\n   TOTAL DIGIT-
LINES: %d\n",total_lines,total_errors,total_digits);
    return 0;
}

void* analyze(void* arg) { //συνάρτηση που αναλύει κάθε αρχείο
    file_thread *thread = (file_thread *)arg; //cast
    int fd = open(thread->filename,O_RDONLY); //το ανοίγω μόνο για
ανάγνωση
    if (fd==-1) {error("open"); pthread_exit(NULL);} //σφάλμα ανοίγματος

    char c;
    ssize_t b = read(fd, &c, 1); //διάβασμα ενός χαρακτήρα (αν υπάρχει)
για έλεγχο

    if (b == 0) {
        // το αρχείο είναι κενό - δε διαβάστηκε χαρακτήρας
        printf("File %s is empty\n", thread->filename);
        close(fd); //κλείσιμο για ασφάλεια
        pthread_exit(NULL);
    }
    if (b == -1) {
        // σφάλμα ανάγνωσης
        error("read");
        close(fd);
        pthread_exit(NULL);
    }
    lseek(fd, 0, SEEK_SET); // επιστροφή στην αρχή του αρχείου

    int lines = 0;int line_errors = 0;int line_with_digits = 0;
    int flag_digits = 0;int i =0;
```

```
    int bufs=BUFFER_SIZE; //για να επεκτείνω μέγεθος δυναμικά
    char *buffer = malloc(BUFFER_SIZE); // δυναμικός πίνακας
    if (!buffer) { error("malloc"); close(fd); pthread_exit(NULL); }
//έλεγχος για μη επιτυχή malloc
    while (b > 0) { //οσο έχει μέσα bytes να διαβάσει
        if (isdigit(c)) flag_digits = 1; // αν ο χαρακτήρας είναι ψηφίο

        if (i>=bufs-1) { // αν φτάσει στο όριο
            bufs*=2; //διπλασιασμός προκαθορισμένου μεγέθους
            char *temp = realloc(buffer,bufs); // μεγαλύτερο μέγεθος με
realloc σε temp για ασφάλεια
            if (!temp) { error("realloc"); free(buffer); close(fd);
pthread_exit(NULL); } //έλεγχος για realloc
            buffer = temp; //θέτω τον πίνακά μας ίσο με το temp - η
realloc διατηρεί τα προηγούμενα δεδομένα στον καινούργιο-μεγαλύτερο
buffer
        }
        if (c != '\n') { //αν δεν είναι αλλαγή γραμμής βάζω χαρακτήρα
στο buffer
            buffer[i++]=c;
        }

        if (c == '\n') { //αλλαγή γραμμής

line_change(buffer,i,flag_digits,&lines,&line_with_digits,&line_errors);
            //επαναφορά για επόμενη επανάληψη
            flag_digits = 0;
            i=0;
            bufs=BUFFER_SIZE;
        }
        b = read(fd, &c, 1); //διάβασμα επόμενου χαρακτήρα
    }
    if (i>0) { // για να μη μου ξεφύγει καμία γραμμή - έχει χαρακτήρες
στο buffer
        // μπορεί μια γραμμή να μη τελειώνει σε \n αλλά πχ με EOF
        // αν υπάρχουν χαρακτήρες στο buffer τους επεξεργαζόμαστε ως
γραμμή

line_change(buffer,i,flag_digits,&lines,&line_with_digits,&line_errors);
    }
    free(buffer); // ελευθέρωση για να μην έχω memory leaks
    close(fd); //κλείσιμο αρχείου
    // θέτω τα δεδομένα στο τρέχων αρχείο μας μέσω του struct
    thread->lines=lines;
    thread->errors=line_errors;
    thread->digits=line_with_digits;
    pthread_exit(NULL); // Τέλος
}

void error(const char *msg) { //διαχείριση με p error
    perror(msg);
}

void line_change(char *buffer,const int i,const int flag_digits, int
*lines, int *digits, int *errors) { // συνάρτηση για την αλλαγή γραμμής
    buffer[i]='\0'; // Τέλος γραμμής - string
    (*lines)++; //αυξάνω γραμμή
    if (flag_digits) {
        (*digits)++; //περιέχει ψηφίο η γραμμή
    }
```

```
    if (strstr(buffer, "ERROR")!=NULL) {
        (*errors)++; // το 'ERROR' είναι substring του ολόκληρου string-γραμμής
    }
}

void print_each_log_stats(file_thread *file,int *lines,int *errors,int *digits) {
    //συνάρτηση που εκτυπώνει για ένα αρχείο τα αποτελέσματα και παράλληλα
    //αυξάνει τους συνολικούς μετρητές με τα τρέχοντα αποτελέσματα του αρχείου για τα συνολικά στατιστικά στο τέλος
    printf("File: %s | Lines: %d | Errors: %d | Lines with digit: %d\n",
        file->filename, file->lines, file->errors, file->digits);
    //αύξηση μετρητών
    (*lines) += file->lines;
    (*errors) += file->errors;
    (*digits) += file->digits;
}
```

Χρησιμοποιείται ίδια λογική με το μέρος Ε, εκεί όπου γίνεται ανάλυση ενός μόνου αρχείου log. Για κάθε αρχείο δημιουργείται ένα ξεχωριστό thread, το οποίο ανοίγει το αρχείο μόνο για ανάγνωση και το διαβάζει χαρακτήρα-χαρακτήρα. Κατά την ανάγνωση, ο κώδικας συγκεντρώνει στατιστικά για κάθε γραμμή: μετράει το συνολικό αριθμό γραμμών κάθε αρχείου, πόσες γραμμές περιέχουν τουλάχιστον ένα ψηφίο (Δε το ζητά ξεκάθαρα το μέρος Ζ, αλλά εγώ το προσθέτω όπως στο μέρος Ε) και πόσες περιέχουν τη λέξη "ERROR". Σε αυτό το μέρος πρόσθεσα δυναμική αύξηση του πίνακα, διότι μπορεί να έχουμε μεγαλύτερες γραμμές σε κάποια αρχεία. Χειρίζεται γραμμές οποιουδήποτε μήκους με δυναμική επέκταση buffer , ώστε να μην υπάρχουν προβλήματα αν μια γραμμή είναι μεγαλύτερη από το αρχικό μέγεθος buffer 1024. Ο κώδικας επίσης περιλαμβάνει έλεγχο σφαλμάτων για αποτυχία ανοίγματος αρχείου, ανάγνωσης ή κατανομής μνήμης, εξασφαλίζοντας ασφαλή εκτέλεση όπως ακριβώς στο μέρος Ε. Στο τέλος με pthread_join, κάθε thread ενημερώνει ένα struct με τα αποτελέσματα του αρχείου, και η main εκτυπώνει τα στατιστικά για κάθε αρχείο ξεχωριστά χρησιμοποιώντας την συνάρτηση print_each_log_stats(), καθώς και συνολικά για όλα τα αρχεία.

Παράδειγμα εκτέλεσης με 4 αρχεία .log :

```
pashalis@pashalis:~/OS/monitor$ gcc parallel_analyze.c -o parallel_analyze
pashalis@pashalis:~/OS/monitor$ ./parallel_analyze raw/network.log raw/system.log raw/security.log raw/timestamps.log
File: raw/network.log | Lines: 6 | Errors: 1 | Lines with digit: 3
File: raw/system.log | Lines: 7 | Errors: 2 | Lines with digit: 3
File: raw/security.log | Lines: 6 | Errors: 1 | Lines with digit: 4
File: raw/timestamps.log | Lines: 25 | Errors: 0 | Lines with digit: 25
    TOTAL LINES: 44
   TOTAL ERRORS: 4
   TOTAL DIGIT-LINES: 35
pashalis@pashalis:~/OS/monitor$
```